Generative Adversarial Networks (GAN)

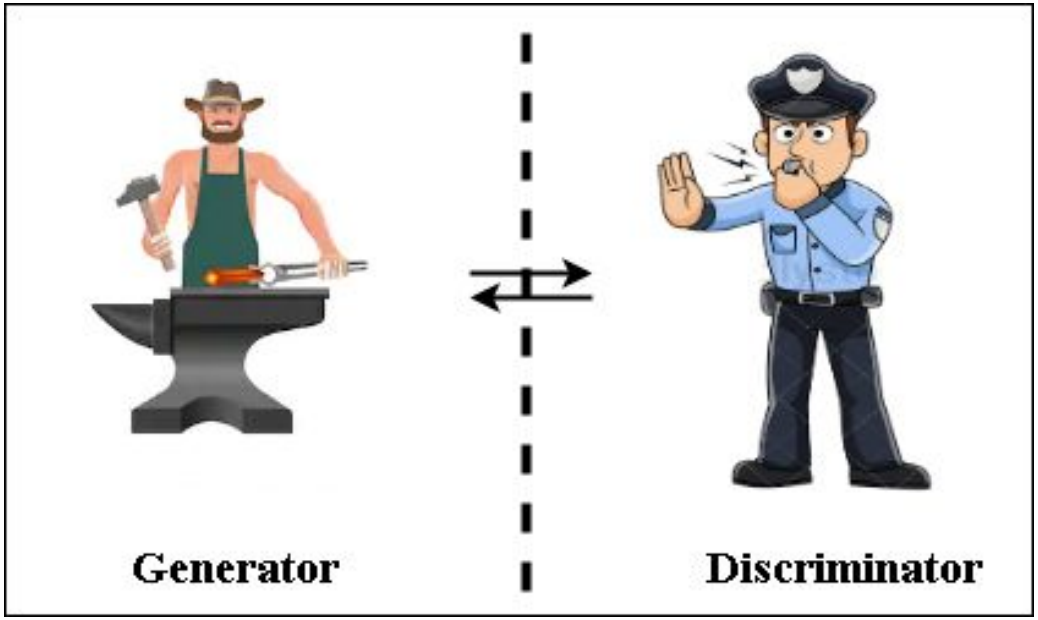
``the most interesting idea in the last 10 years in machine learning." -Yann Le Cun

Introduction

- **Generative v/s discriminative models:**

- Generative models: model the distribution of individual classes.

- Discriminative models: learn the boundary between the classes.



Introduction

- GAN: Generative Adversarial Networks.
- Generator + Discriminator.
- Generative: To generate the samples based on training data.
- Adversarial: The training of the model is done in adversarial manner.

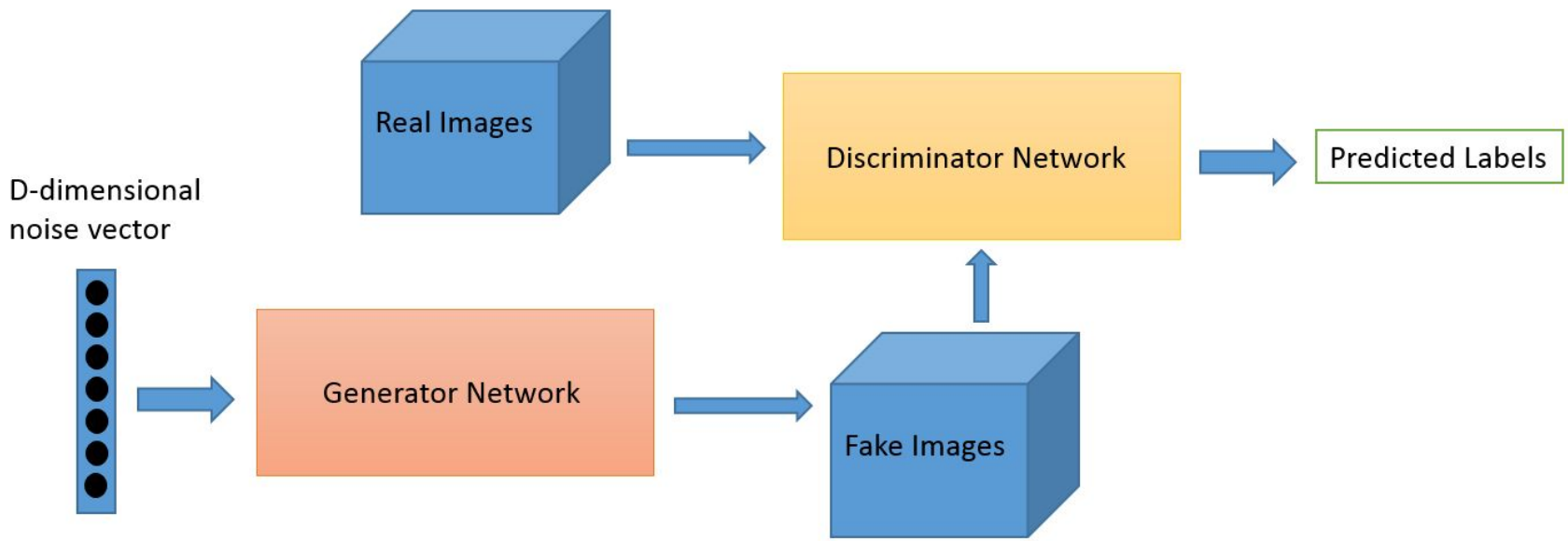
  - Generator (G) and Discriminator (D) work against each other.
- Networks: Deep neural networks.

Ref: Ian J. Goodfellow et al.,Generative Adversarial Networks,arXiv:1406.2661v1,2014.

Introduction

(a) Generator G takes some random noise and tries to produce real input training images by fooling the discriminator D.

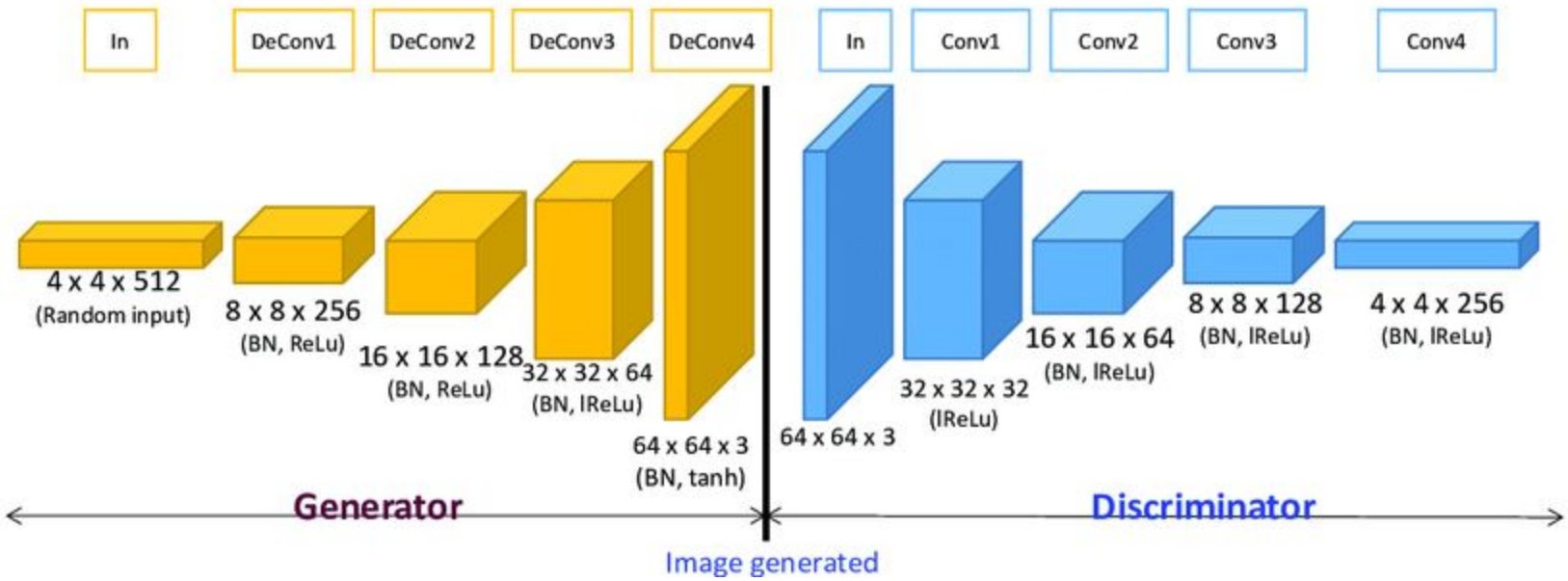
(b) Discriminator D tries to classify the input being real if it's coming from the training data and fake if it's coming from the generator G.



Ref: https://skymind.ai/wiki/generative-adversarial-network-gan

Deep Convolutional (DCGAN)

- Deep Convolutional networks are used for both generator (G) and discriminator (D).
- Batch normalization is done in both networks.



Ref: Suh, Sungho & Lee, Haebom & Jo, Jun & Lukowicz, Paul & Lee, Yong. (2019). Generative Oversampling Method for Imbalanced Data on Bearing Fault Detection and Diagnosis. Applied Sciences.

How it works?

Let's consider:

z: Noise vector
$G(z)$: Generator's output $\rightarrow x_{fake}$
x: training sample $\rightarrow x_{real}$
$D(x)$: Discriminator's output for $x_{real} \rightarrow P(y|x_{real}) \rightarrow \{0,1\}$
$D(G(z))$: Discriminator's output for $x_{fake} \rightarrow P(y|x_{fake}) \rightarrow \{0,1\}$

at Discriminator D	at Generator G
D(x) $\rightarrow$ should be maximized D(G(z)) $\rightarrow$ should be minimized	D(G(z)) $\rightarrow$ should be maximized

- Discriminator: should maximize the real data while minimizing the fake data.
- Generator: should maximize the fake data.

Ref: Ian J. Goodfellow et al.,Generative Adversarial Networks,arXiv:1406.2661v1,2014.

How it works?

Let's consider:

z: Noise vector
$G(z)$: Generator's output $\rightarrow x_{fake}$
x: training sample $\rightarrow x_{real}$
$D(x)$: Discriminator's output for x_{real}
$D(G(z))$: Discriminator's output for x_{fake}

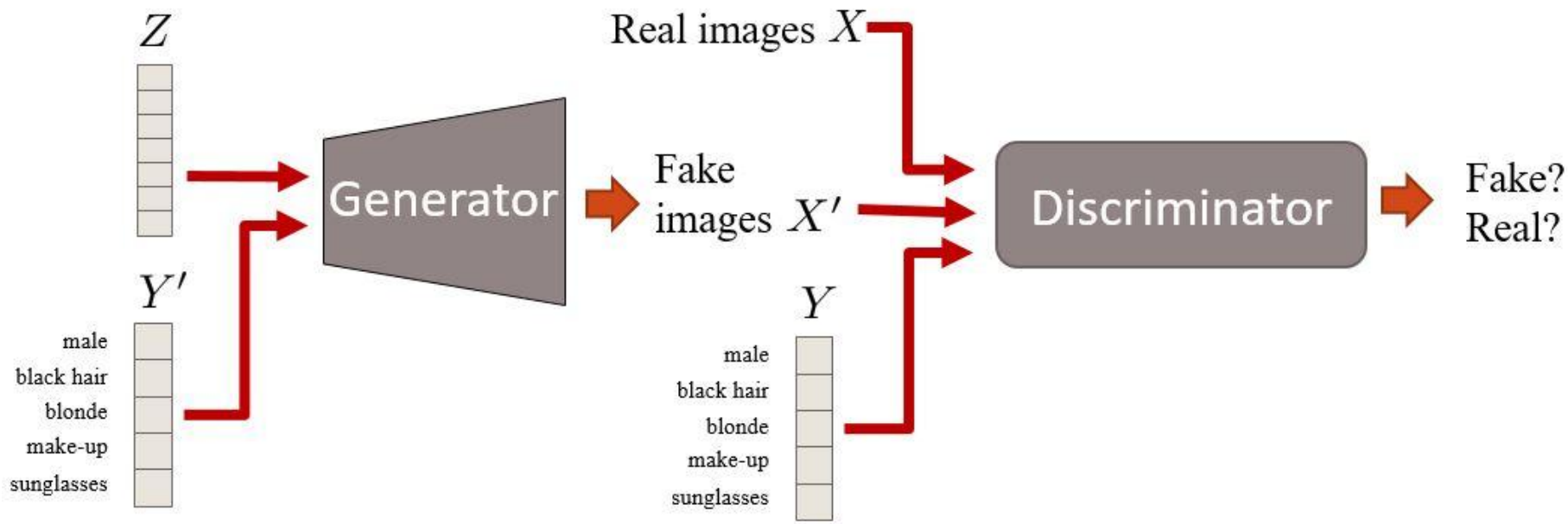
$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}(x)}[logD(x)] + \mathbb{E}_{z\sim p_z(z)}[log(1-D(G(z)))]$$

$$\max_D V(D) = \underbrace{\mathbb{E}_{x\sim p_{data}(x)}[logD(x)]}_{\text{recognize real images better}} + \underbrace{\mathbb{E}_{z\sim p_z(z)}[log(1-D(G(z)))]}_{\text{recognize fake images better}}$$

$$\min_G V(G) = \mathbb{E}_{z\sim p_z(z)}[log(1-D(G(z)))]$$

Ref: Ian J. Goodfellow et al.,Generative Adversarial Networks,arXiv:1406.2661v1,2014.

Conditional GAN



- At Discriminator D, given input x and class y → **classifies the input image is fake or real.**
- At Generator G, given noise z, and class y → **generates the image which is conditioned by y.**

Ref: Mirza et al, "Conditional Generative Adversarial Nets", https://arxiv.org/abs/1411.1784.

Conditional GAN

- If both the **generator** and **discriminator** are conditioned on some extra information **y**.
- **y** can typically be a class label or tag.

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x}|\boldsymbol{y})] + \mathbb{E}_{\boldsymbol{z}\sim p_z(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z}|\boldsymbol{y})))]$$

$$L_D^{CGAN} = E\big[\log\big(D(x,c)\big)\big] + E\big[\log\big(1 - D(G(z),c)\big)\big]$$

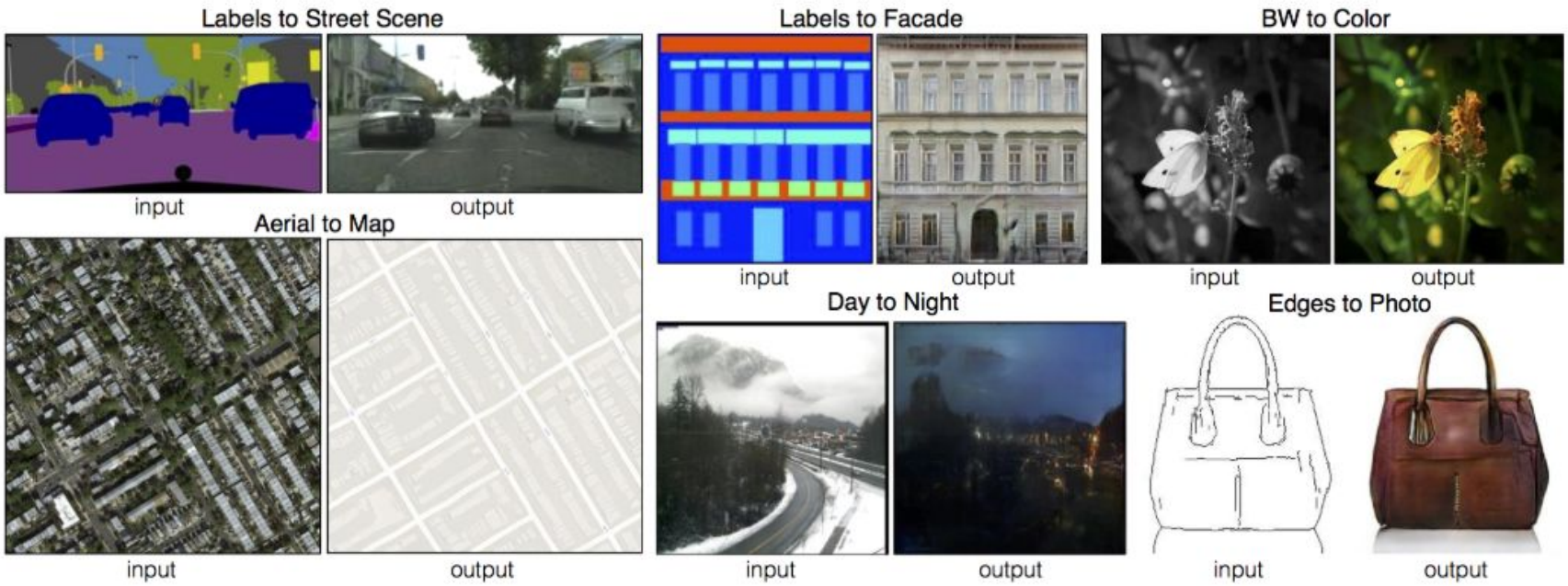
$$L_G^{CGAN} = E\big[\log\big(D(G(z),c)\big)\big]$$

c is the condition = y

Ref: Mirza et al, “Conditional Generative Adversarial Nets”, https://arxiv.org/abs/1411.1784.

Pix2Pix GAN

- "Pixel to Pixel" which means , in an image it takes a pixel, then convert that into another pixel.
- the goal is to learn the mapping from an input image to an output image.



Ref: Isola et al, "Image-to-Image Translation with Conditional Adversarial Networks", https://arxiv.org/abs/1611.07004v3.

Pix2Pix

- Training data pairs (**x** and **y** where **x** is the input image and **y** is the output image)
- Pix2Pix uses the **conditional GAN (CGAN)** → G : {x, z} → y.

  (z → noise vector, x → input image, y → output image)
- Generator Network(Encode- decode architecture) as an image is the input , we want to learn the deep representation and decode it
- CGAN loss function and L1 or L2 distance.

$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x,y}[\log D(x, y)] + \mathbb{E}_{x,z}[\log(1 - D(x, G(x, z)))],$$

$$\mathcal{L}_{L1}(G) = \mathbb{E}_{x,y,z}[\|y - G(x, z)\|_1]. \qquad \mathcal{L}_{L_2}(G) = \mathbb{E}_{x,y,z}\left(y - G(x, z)\right)^2$$

Ref: Isola et al, “Image-to-Image Translation with Conditional Adversarial Networks”, https://arxiv.org/abs/1611.07004v3.

Pix2Pix

- Training pix2pix Gan as same as training any GAN except a little modification at the generator's loss function.
- The generator G is not only trying to reduce the loss from discriminator but also trying to move the fake distribution close to real distribution by using L1 or L2 loss
- The loss function of generator network is:

$$G^* = \arg\min_G \max_D \mathcal{L}_{cGAN}(G, D) + \lambda \mathcal{L}_{L1}(G)$$

CycleGAN

- Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks.
- Given any two unordered image collections X and Y , the new algorithm learns to automatically “translate” an image from one into the other and vice versa.

- In above figure, both horse (x) and zebra (y) can be translated to other one.

Ref: Zhu et al, Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks, https://arxiv.org/abs/1703.10593v6

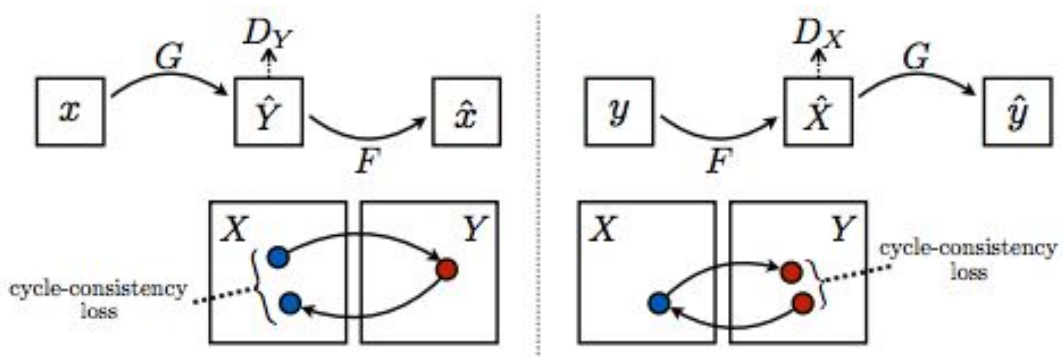
CycleGAN

- Pix2Pix: In a paired dataset ⟶ **x** and **y** share some common features ⟶ making of neural nets that do mapping between the both images with the loss (L1).
- Cycle GAN: an unpaired dataset ,there may no meaningful transformation that we can learn from x to y ⟶ ??? How to learn the mapping between x and y, for image to image translation.

Ref: Zhu et al, Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks, https://arxiv.org/abs/1703.10593v6

CycleGAN

- The training data consists of two different sets of images. One set of images called "**Domain X**", another set of images called "**Domain Y**".
- E.g. one set full of horses while another set full of zebras (no ordered pairing).
- → Two Generators with two different functions/processes (G(x), and F(G(x))) and Two Discriminators.
- → GAN loss along with Cycle Consistent loss (new).



Ref: Zhu et al, Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks, https://arxiv.org/abs/1703.10593v6

CycleGAN

- The loss function of cycleGAN is:

$$\begin{aligned}\mathcal{L}_{\text{cyc}}(G,F) =& \mathbb{E}_{x\sim p_{\text{data}}(x)}[\|F(G(x))-x\|_1] \\ &+\mathbb{E}_{y\sim p_{\text{data}}(y)}[\|G(F(y))-y\|_1].\end{aligned}$$

- The final loss = GAN loss + Cycle loss

$$\begin{aligned}\mathcal{L}(G,F,D_X,D_Y) =& \mathcal{L}_{\text{GAN}}(G,D_Y,X,Y) \\ &+\mathcal{L}_{\text{GAN}}(F,D_X,Y,X) \\ &+\lambda\mathcal{L}_{\text{cyc}}(G,F),\end{aligned}$$

- Here is the optimal objective function for cycle GAN

$$G^*,F^* = \arg\min_{G,F}\max_{D_x,D_Y}\mathcal{L}(G,F,D_X,D_Y)$$

GAN: Types

1. Deep Convolutional GANs (DCGANs)
2. Conditional GAN
3. Pix2pix GAN
4. Cycle GAN

5. Super Resolution GANs (SRGANs)
6. Texture GAN
7. GAN-art
8. Progressive-GAN
9. Stack GAN

and many more…